Princess Nourah Bint Abdulrahman University

ETL and ELT pipelines for Gaming Dataset

GroupID: 4

By

Students ID	Students Name	Section
445008596	Dana Almkhlafi	64s

This project is part of Assessments Grades

for the Course DS437: Data Engineering

CCIS, PNU

Riyadh, KSA

Second Semester 2026-2027

Princess Nourah Bint Abdulrahman University

Table of Contents

Chapter 1: Introduction

Data engineering is important in modern systems because companies collect large amounts of data from different sources. Raw data is usually not ready for analysis because it may contain missing values, duplicated records, or inconsistent formats. Therefore, data pipelines are used to extract, clean, transform, and store data in an organized way.[1]

In this project, we implemented both ETL (Extract, Transform, Load) and ELT (Extract, Load, Transform) pipelines using a video games dataset. The dataset includes information such as game names, platforms, genres, publishers, release years, and global sales. We also used the RAWG API to collect additional information like ratings, Metacritic scores, and playtime.

The main purpose of this project is to compare ETL and ELT pipelines in terms of performance, scalability, efficiency, and data quality using practical implementation with Python, SQLite, PostgreSQL, and SQL.

1.1 Background and motivation

Data engineering helps organizations prepare raw data for analysis and decision-making. Modern systems collect data from APIs, databases, and CSV files, but this data is often incomplete or inconsistent. Because of this, data cleaning and transformation are necessary before using data analysis.

The gaming industry produces a large amount of data related to game sales, ratings, publishers, and platforms. This data can provide useful insights, but it must first be processed and organized correctly.

In this project, we implemented ETL and ELT pipelines to understand how each approach handles data extraction, transformation, validation, and storage. We also compared the advantages and limitations of both methods to identify the best use cases for each one.

1.2 Problem statement

As data volume increases, organizations face challenges in processing and integrating data from multiple sources. It is also difficult to decide whether ETL or ELT is more suitable for handling large datasets and different data formats.[2]

The video games dataset used in this project contains missing values, duplicate records, inconsistent formats, and incomplete information collected from CSV files and APIs. These problems reduce data quality and affect the accuracy of analysis.

Therefore, this project focuses on designing ETL and ELT pipelines that can extract, clean, transform, validate, and store gaming data in a structured format suitable for analysis and visualization.

1.3 Objectives of the Data engineering Project

The objectives of this project are:

- Design and implement an ETL pipeline for video games data.
- Design and implement an ELT pipeline using PostgreSQL.
- Extract gaming data from CSV files, APIs, and databases.
- Clean and transform the data to improve quality and consistency.

- Store processed data in SQLite and PostgreSQL databases.
- Compare ETL and ELT in terms of performance, scalability, and efficiency.
- Visualize the processed data using BI dashboards.

1.4 Sustainable Development Goals (SDGs)

This project supports SDG 9: Industry, Innovation, and Infrastructure [3] by implementing modern data engineering technologies such as ETL and ELT pipelines, APIs, databases, and cloud data warehouses to improve data processing and management.
The project also supports SDG 8: Decent Work and Economic Growth [3] by improving data workflows and enabling efficient data analysis for better decision-making.

Chapter 2 : System Design and Architecture

This chapter explains the system architecture used to implement the ETL and ELT pipelines. The project combines multiple data sources and applies ingestion, cleaning, transformation, and storage processes to prepare the data for analysis.

2.1 Data sources and collection methods

In this project, different data sources were used for ETL and ELT pipelines.

ETL Pipeline

Two main sources were used:

- CSV file (Video_Games_500.csv):
  This dataset contains information about video games such as game names, platforms, genres, publishers, release years, and global sales.
- RAWG API:
  The API was used to collect additional game details such as ratings, Metacritic scores, release dates, and playtime.

ELT Pipeline

The following sources were used:

- SQLite Database:
  Contains developer information such as developer names, headquarters, and foundation years.
- CSV file (Video_Games.csv):
  Contains video games data including platform, genre, publisher, and global sales.

These sources were selected to simulate real-world data engineering scenarios using APIs, CSV files, and relational databases.

2.2 Data Ingestion method

In this project, batch ingestion was used to collect and process data.

ETL Pipeline

- Data was extracted from the CSV file using pandas.
- Additional data was collected from the RAWG API using requests.
- The datasets were merged and transformed before loading into SQLite.

ELT Pipeline

- Raw data from the SQLite database and CSV file were loaded directly into PostgreSQL staging tables.
- Data transformation was performed later inside PostgreSQL using SQL queries.

2.3 Data cleaning and transformation methods

Several cleaning and transformation methods were applied in both ETL and ELT pipelines.

ETL Pipeline

The data was cleaned before loading using Python and pandas. The following steps were applied:

- Cleaning text values and removing extra spaces.
- Replacing invalid text values with missing values.
- Handling missing values by removing rows with missing critical columns and filling non-critical values.
- Converting numeric columns into suitable numeric data types.
- Converting release year values into integer format.
- Removing duplicate records based on game name and platform.
- Merging CSV and RAWG API data into one dataset using a left join.
- Resetting the dataset index after cleaning.

After transformation, the cleaned dataset was loaded into SQLite.

ELT Pipeline

In the ELT workflow, raw data was loaded first into PostgreSQL, and transformation was done later using SQL queries.

The transformations included:

- Removing duplicate records using SELECT DISTINCT.
- Cleaning text using TRIM().
- Standardizing platform names using UPPER().
- Replacing missing values using COALESCE().
- Converting release year values into integers using CAST().

These transformations improved data quality and prepared the final tables for dashboard analysis and visualization.

Chapter 3: ETL Pipeline Design

3.1 Data pipeline architectute(ETL)

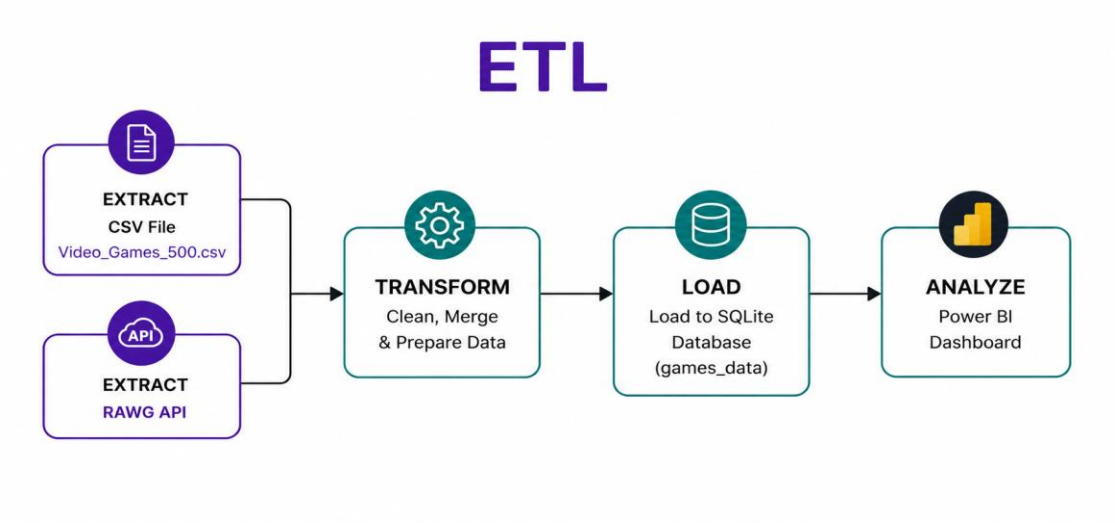


Figure 1 ETL pipeline architecture

3.2 Extract: Collecting data from sources (APIs, databases, files)

In this phase, data was collected from two sources:

- A CSV file (**Video_Games_500_dirty.csv**) containing video games data.
- The **RAWG API** for additional game information such as ratings and release dates.

Python libraries such as pandas, requests, and sqlite3 were used during the extraction process

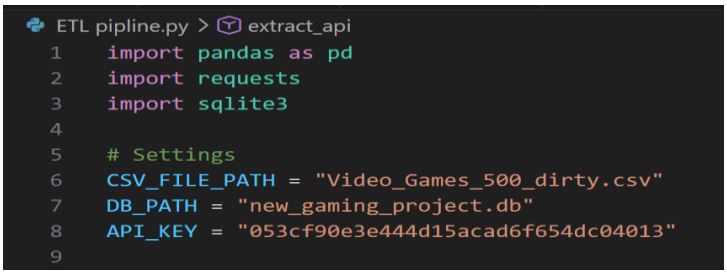


Figure 2 Import data

```
# --- 1. Extract CSV data ---
def extract_csv(file_path):
    df = pd.read_csv(file_path)

    # Clean column names
    df.columns = df.columns.str.strip().str.upper()

    print("CSV extracted.")
    return df
```

Figure 3 Extract CSV

```python
# --- 2. Extract API data ---
def extract_api(names):
    results = []

    for name in names:
        url = "https://api.rawg.io/api/games"

        params = {
            "key": API_KEY,
            "search": name,
            "page_size": 1
        }

        try:
            response = requests.get(url, params=params, timeout=10)
            data = response.json()

            if data.get("results"):
                game = data["results"][0]

                results.append({
                    "TITLE": name,
                    "RAWG_NAME": game.get("name"),
                    "RAWG_RATING": game.get("rating"),
                    "METACRITIC": game.get("metacritic"),
                    "RAWG_RELEASED": game.get("released"),
                    "RAWG_PLAYTIME": game.get("playtime")
                })

        except:
            print("API skipped:", name)

    print("API extracted.")
    return pd.DataFrame(results)
```

Figure 4 Extract API

3.3 Transform: Cleaning, filtering, aggregating data before loading

After extraction, several transformation steps were applied to clean and prepare the data before loading:

- Missing values in ratings, scores, and playtime columns were replaced with suitable values.
- Numeric columns were converted into appropriate data types.
- Duplicate rows were removed based on game name and platform.
- The release year column was converted into integer format.
- Data from the CSV file and RAWG API were merged into one unified dataset.

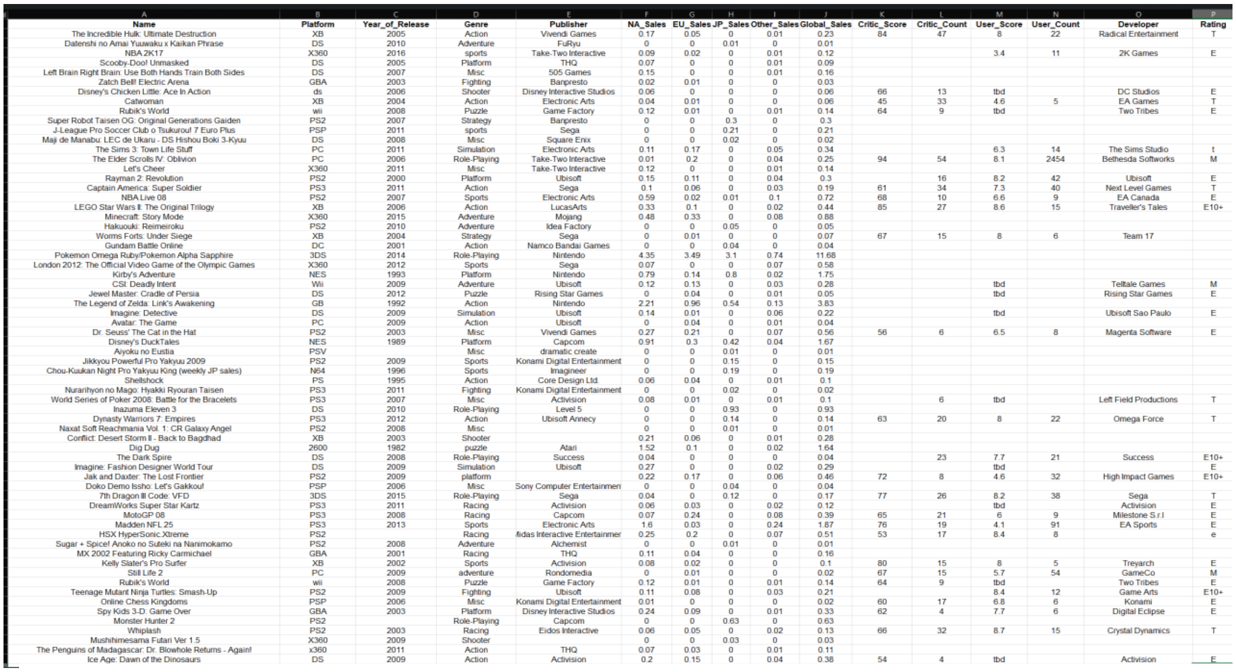
- The dataset index was reset after cleaning to maintain proper structure.

Finally, the cleaned and merged dataset was prepared successfully before loading into the SQLite database.

ETL pipline.py > transform

```python
def transform(csv_df, api_df):
    # Fill API numeric values only with 0 because API may not return them
    api_numeric_fill = [
        "RAWG_RATING",
        "METACRITIC",
        "RAWG_PLAYTIME"
    ]

    for col in api_numeric_fill:
        if col in df.columns:
            df[col] = df[col].fillna(0)

    # Fill optional score/count values with 0
    optional_numeric_fill = [
        "CRITIC_SCORE",
        "CRITIC_COUNT",
        "USER_SCORE",
        "USER_COUNT"
    ]

    for col in optional_numeric_fill:
        if col in df.columns:
            df[col] = df[col].fillna(0)

    # Convert year to integer
    df["YEAR_OF_RELEASE"] = df["YEAR_OF_RELEASE"].astype(int)

    # Remove duplicates
    df.drop_duplicates(
        subset=["NAME", "PLATFORM"],
        inplace=True
    )

    # Reset index
    df.reset_index(drop=True, inplace=True)
```

Figure 5 Data Transformation



Name	Platform	Year_of_Release	Genre	Publisher	NA_Sales	EU_Sales	JP_Sales	Other_Sales	Global_Sales	Critic_Score	Critic_Count	User_Score	User_Count	Developer	Rating
The Incredible Hulk: Ultimate Destruction	XB	2005	Action	Vivendi Games	0.17	0.05	0	0.01	0.23	84	47	8	22	Radical Entertainment	T
Datenshi no Amai Yuuwaku x Kaikan Phrase	DS	2010	Adventure	FuRyu	0	0	0.01	0	0.01						
NBA 2K17	X360	2016	sports	Take-Two Interactive	0.09	0.02	0	0.01	0.12			3.4	11	2K Games	E
Scooby-Doo! Unmasked	DS	2005	Platform	THQ	0.07	0	0	0.01	0.09						
Left Brain Right Brain: Use Both Hands Train Both Sides	DS	2007	Misc	505 Games	0.15	0	0	0.01	0.16						
Zatch Bell! Electric Arena	GBA	2003	Fighting	Banpresto	0.02	0.01	0	0	0.03						
Disney's Chicken Little: Ace In Action	ds	2006	Shooter	Disney Interactive Studios	0.06	0	0	0	0.06	66	13	tbd		DC Studios	E
Catwoman	XB	2004	Action	Electronic Arts	0.04	0.01	0	0	0.06	45	33	4.6	5	EA Games	T
Rubik's World	wii	2008	Puzzle	Game Factory	0.12	0.01	0	0.01	0.14	64	9	tbd		Two Tribes	E
Super Robot Taisen OG: Original Generations Gaiden	PS2	2007	Strategy	Banpresto	0	0	0.3	0	0.3						
J-League Pro Soccer Club o Tsukurou! 7 Euro Plus	PSP	2011	sports	Sega	0	0	0.21	0	0.21						
Maji de Manabu: LEC de Ukaru - DS Hishou Boki 3-Kyuu	DS	2008	Misc	Square Enix	0	0	0.02	0	0.02						
The Sims 3: Town Life Stuff	PC	2011	Simulation	Electronic Arts	0.11	0.17	0	0.05	0.34			6.3	14	The Sims Studio	t
The Elder Scrolls IV: Oblivion	PC	2006	Role-Playing	Take-Two Interactive	0.01	0.2	0	0.04	0.25	94	54	8.1	2454	Bethesda Softworks	M
Let's Cheer	X360	2011	Misc	Take-Two Interactive	0.12	0	0	0.01	0.14						
Rayman 2: Revolution	PS2	2000	Platform	Ubisoft	0.15	0.11	0	0.04	0.3		16	8.2	42	Ubisoft	E
Captain America: Super Soldier	PS3	2011	Action	Sega	0.1	0.06	0	0.03	0.19	61	34	7.3	40	Next Level Games	T
NBA Live 08	PS2	2007	Sports	Electronic Arts	0.59	0.02	0.01	0.1	0.72	68	10	6.6	9	EA Canada	E
LEGO Star Wars II: The Original Trilogy	XB	2006	Action	LucasArts	0.33	0.1	0	0.02	0.44	85	27	8.6	15	Traveller's Tales	E10+
Minecraft: Story Mode	X360	2015	Adventure	Mojang	0.48	0.33	0	0.08	0.88						
Hakuouki: Reimeiroku	PS2	2010	Adventure	Idea Factory	0	0	0.05	0	0.05						
Worms Forts: Under Siege	XB	2004	Strategy	Sega	0	0.01	0	0	0.07	67	15	8	6	Team 17	
Gundam Battle Online	DC	2001	Action	Namco Bandai Games	0	0	0.04	0	0.04						
Pokemon Omega Ruby/Pokemon Alpha Sapphire	3DS	2014	Role-Playing	Nintendo	4.35	3.49	3.1	0.74	11.68						
London 2012: The Official Video Game of the Olympic Games	X360	2012	Sports	Sega	0.07	0	0	0.07	0.58						
Kirby's Adventure	NES	1993	Platform	Nintendo	0.79	0.14	0.8	0.02	1.75						
CSI: Deadly Intent	Wii	2009	Adventure	Ubisoft	0.12	0.13	0	0.03	0.28			tbd		Telltale Games	M
Jewel Master: Cradle of Persia	DS	2012	Puzzle	Rising Star Games	0	0.04	0	0.01	0.05			tbd		Rising Star Games	E
The Legend of Zelda: Link's Awakening	GB	1992	Action	Nintendo	2.21	0.96	0.54	0.13	3.83						
Imagine: Detective	DS	2009	Simulation	Ubisoft	0.14	0.01	0	0.06	0.22			tbd		Ubisoft Sao Paulo	E
Avatar: The Game	PC	2009	Action	Ubisoft	0	0.04	0	0.01	0.04						
Dr. Seuss' The Cat in the Hat	PS2	2003	Misc	Vivendi Games	0.27	0.21	0	0.07	0.56	56	6	6.5	8	Magenta Software	E
Disney's DuckTales	NES	1989	Platform	Capcom	0.91	0.3	0.42	0.04	1.67						
Aiyoku no Eustia	PSV		Misc	dramatic create	0	0	0.01	0	0.01						
Jikkyou Powerful Pro Yakyuu 2009	PS2	2009	Sports	Konami Digital Entertainment	0	0	0.15	0	0.15						
Chou-Kuukan Night Pro Yakyuu King (weekly JP sales)	N64	1996	Sports	Imagineer	0	0	0.19	0	0.19						
Shellshock	PS	1995	Action	Core Design Ltd.	0.06	0.04	0	0.01	0.1						
Nurarihyon no Mago: Hyakki Ryouran Taisen	PS3	2011	Fighting	Konami Digital Entertainment	0	0	0.02	0	0.02						
World Series of Poker 2008: Battle for the Bracelets	PS3	2007	Misc	Activision	0.08	0.01	0	0.01	0.1		6	tbd		Left Field Productions	T
Inazuma Eleven 3	DS	2010	Role-Playing	Level 5	0	0	0.93	0	0.93						
Dynasty Warriors 7: Empires	PS3	2012	Action	Ubisoft Annecy	0	0	0.14	0	0.14	63	20	8	22	Omega Force	T
Naxat Soft Reachmania Vol. 1: CR Galaxy Angel	PS2	2008	Misc		0	0	0.01	0	0.01						
Conflict: Desert Storm II - Back to Bagdhad	XB	2003	Shooter		0.21	0.06	0	0.01	0.28						
Dig Dug	2600	1982	puzzle	Atari	1.52	0.1	0	0.02	1.64						
The Dark Spire	DS	2008	Role-Playing	Success	0.04	0	0	0	0.04		23	7.7	21	Success	E10+
Imagine: Fashion Designer World Tour	DS	2009	Simulation	Ubisoft	0.27	0	0	0.02	0.29			tbd			E
Jak and Daxter: The Lost Frontier	PS2	2009	platform		0.22	0.17	0	0.06	0.46	72	8	4.6	32	High Impact Games	E10+
Doko Demo Issho: Let's Gakkou!	PSP	2006	Misc	Sony Computer Entertainmen	0	0	0.04	0	0.04						
7th Dragon III Code: VFD	3DS	2015	Role-Playing	Sega	0.04	0	0.12	0	0.17	77	26	8.2	38	Sega	T
DreamWorks Super Star Kartz	PS3	2011	Racing	Activision	0.06	0.03	0	0.02	0.12			tbd		Activision	E
MotoGP 08	PS3	2008	Racing	Capcom	0.07	0.24	0	0.08	0.39	65	21	6	9	Milestone S.r.l	E
Madden NFL 25	PS3	2013	Sports	Electronic Arts	1.6	0.03	0	0.24	1.87	76	19	4.1	91	EA Sports	E
HSX HyperSonic.Xtreme	PS2		Racing	Midas Interactive Entertainmer	0.25	0.2	0	0.07	0.51	53	17	8.4	8		e
Sugar + Spice! Anoko no Suteki na Nanimokamo	PS2	2008	Adventure	Alchemist	0	0	0.01	0	0.01						
MX 2002 Featuring Ricky Carmichael	GBA	2001	Racing	THQ	0.11	0.04	0	0	0.16						
Kelly Slater's Pro Surfer	XB	2002	Sports	Activision	0.08	0.02	0	0	0.1	80	15	8	5	Treyarch	E
Still Life 2	PC	2009	adventure	Rondomedia	0	0.01	0	0	0.02	67	15	5.7	54	GameCo	M
Rubik's World	wii	2008	Puzzle	Game Factory	0.12	0.01	0	0.01	0.14	64	9	tbd		Two Tribes	E
Teenage Mutant Ninja Turtles: Smash-Up	PS2	2009	Fighting	Ubisoft	0.11	0.08	0	0.03	0.21			8.4	12	Game Arts	E10+
Online Chess Kingdoms	PSP	2006	Misc	Konami Digital Entertainment	0.01	0	0	0	0.02	60	17	6.8	6	Konami	E
Spy Kids 3-D: Game Over	GBA	2003	Platform	Disney Interactive Studios	0.24	0.09	0	0.01	0.33	62	4	7.7	6	Digital Eclipse	E
Monster Hunter 2	PS2		Role-Playing	Capcom	0	0	0.63	0	0.63						
Whiplash	PS2	2003	Racing	Eidos Interactive	0.06	0.05	0	0.02	0.13	66	32	8.7	15	Crystal Dynamics	T
Mushihimesama Futari Ver 1.5	X360	2009	Shooter		0	0	0.03	0	0.03						
The Penguins of Madagascar: Dr. Blowhole Returns - Again!	x360	2011	Action	THQ	0.07	0.03	0	0.01	0.11						
Ice Age: Dawn of the Dinosaurs	DS	2009	Action	Activision	0.2	0.15	0	0.04	0.38	54	4	tbd		Activision	E

Figure 6 Data before transformation

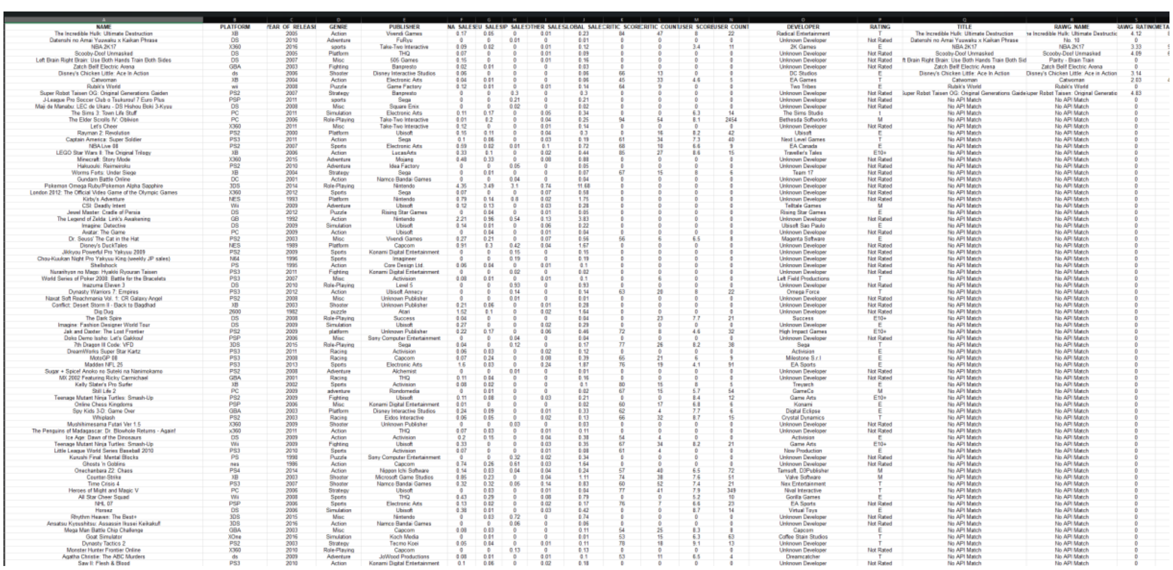


Figure 7 Data after transformation

3.4 Load: Storing transformed data into a data warehouse (e.g.,Sqlite)

After completing the transformation process, the final cleaned dataset was loaded into an SQLite database named **new_gaming_project.db**. The transformed data was stored in a table called **games_data**, which contains the unified dataset collected from both the CSV file and the RAWG API.

The data was loaded using the to_sql() function with the replace option to ensure the table is updated whenever the ETL pipeline is executed again.

Storing the data in SQLite provides a structured format that supports efficient querying, data management, and visualization using tools such as Power BI.

```
# --- 4. Load data to SQLite database ---
def load(df, db):
    conn = sqlite3.connect(db)

    df.to_sql(
        "games_data",
        conn,
        if_exists="replace",
        index=False
    )

    conn.close()

    print("Data loaded to Database.")
```

Figure 8 Data Loading

Chapter 4: Implementation and Testing

4.1 Step-by-step ETL process execution using python program

Python was used to build the ETL (Extract, Transform, Load) pipeline to automate the process of extracting, cleaning, merging, and storing video games data from both the CSV dataset and the RAWG API.

Step 1: Extract Data from CSV:

First, the dataset "Video_Games_500.csv" was loaded using pandas. The dataset includes information about video games such as game title, platform, genre, publisher, sales, and review scores. After importing the file, the column names were standardized to maintain consistency before merging.

Step 2: Extract Data from API:

Then, additional game details were collected from the RAWG API using the game titles available in the CSV dataset. The API returned extra information including RAWG rating, Metacritic score, release date, and game playtime. The extracted API records were stored in a separate pandas DataFrame.

Step 3: Merge and Clean the Data:

After extraction, the CSV data and API data were combined into a single unified DataFrame based on the game title. Several data cleaning techniques were applied, such as handling missing values, converting columns into appropriate numeric formats, removing duplicate records, and resetting the index to improve data quality and consistency.

Step 4: Load Cleaned Data to SQLite:

Finally, the transformed dataset was loaded into an SQLite database named "new_gaming_project.db". The final data was stored in the "games_data" table, which can be used later for SQL queries, dashboard creation, and data visualization using Power BI.

```
C:\Users\danhs\venv\sqlite>sqlite3 "C:\Users\danhs\venv\new_gaming_project.db"
SQLite version 3.51.2 2026-01-09 17:27:48
Enter ".help" for usage hints.
sqlite> .tables
games_data
sqlite> .mode column
sqlite> .headers on
sqlite> SELECT * FROM games_data LIMIT 10;
NAME                                               PLATFORM  YEAR_OF_RELEASE  GENRE      PUBLISHER                NA_SALES  EU_SALES  JP_SA
LES  OTHER_SALES  GLOBAL_SALES  CRITIC_SCORE  CRITIC_COUNT  USER_SCORE  USER_COUNT  DEVELOPER            RATING     TITLE
                        RAWG_NAME                                RAWG_RATING  METACRITIC  RAWG_RELEASED  RAWG_PLAYTIME
---------------------------------------------------  --------  ---------------  ---------  -----------------------  --------  --------  -----
---  ----------  ------------  ------------  ------------  ----------  ----------  -------------------  ---------  ----------  ----------
----------------------  ----------------------------------------  ----------  ----------  -------------  -------------
The Incredible Hulk: Ultimate Destruction          XB        2005             Action     Vivendi Games            0.17      0.05      0.0
     0.01         0.23          84.0          47.0          8.0          22.0        Radical Entertainment  T          The Incredible Hulk: Ultimate
Destruction              The Incredible Hulk: Ultimate Destruction  4.12         84.0        2005-08-23     0.0
Datenshi no Amai Yuuwaku x Kaikan Phrase           DS        2010             Adventure  FuRyu                    0.0       0.0       0.01
     0.0          0.01          0.0           0.0           0.0          0.0         Unknown Developer      Not Rated  Datenshi no Amai Yuuwaku x Kai
kan Phrase               No. 10                                   0.0         0.0          2019-07-19     0.0
NBA 2K17                                           X360      2016             sports     Take-Two Interactive     0.09      0.02      0.0
     0.01         0.12          0.0           0.0           3.4          11.0        2K Games                 E          NBA 2K17
                         NBA 2K17                                 3.33        90.0         2016-09-16     14.0
Scooby-Doo! Unmasked                               DS        2005             Platform   THQ                      0.07      0.0       0.0
     0.01         0.09          0.0           0.0           0.0          0.0         Unknown Developer      Not Rated  Scooby-Doo! Unmasked
                         Scooby-Doo! Unmasked                     4.09        61.0         2005-09-12     0.0
Left Brain Right Brain: Use Both Hands Train Both Sides  DS        2007             Misc       505 Games                0.15      0.0       0.0
     0.01         0.16          0.0           0.0           0.0          0.0         Unknown Developer      Not Rated  Left Brain Right Brain: Use Bo
th Hands Train Both Sides  Parity - Brain Train                     0.0         0.0          2016-01-06     0.0
Zatch Bell! Electric Arena                         GBA       2003             Fighting   Banpresto                0.02      0.01      0.0
     0.0          0.03          0.0           0.0           0.0          0.0         Unknown Developer      Not Rated  Zatch Bell! Electric Arena
                         Zatch Bell! Electric Arena               0.0         0.0          2003-12-12     0.0
Disney's Chicken Little: Ace In Action             ds        2006             Shooter    Disney Interactive Studios  0.06      0.0       0.0
     0.0          0.06          66.0          13.0          0.0          0.0         DC Studios               E          Disney's Chicken Little: Ace I
n Action                 Disney's Chicken Little: Ace in Action     3.14         0.0          2006-09-08     1.0
Catwoman                                           XB        2004             Action     Electronic Arts          0.04      0.01      0.0
```

```
sqlite> SELECT NAME, PLATFORM, GENRE, RAWG_RATING
   ...> FROM games_data
   ...> LIMIT 10;
NAME                                                PLATFORM  GENRE       RAWG_RATING
--------------------------------------------------  --------  ----------  -----------
The Incredible Hulk: Ultimate Destruction           XB        Action      4.12
Datenshi no Amai Yuuwaku x Kaikan Phrase            DS        Adventure   0.0
NBA 2K17                                            X360      sports      3.33
Scooby-Doo! Unmasked                                DS        Platform    4.09
Left Brain Right Brain: Use Both Hands Train Both Sides  DS        Misc        0.0
Zatch Bell! Electric Arena                          GBA       Fighting    0.0
Disney's Chicken Little: Ace In Action              ds        Shooter     3.14
Catwoman                                            XB        Action      2.03
Rubik's World                                       wii       Puzzle      0.0
Super Robot Taisen OG: Original Generations Gaiden  PS2       Strategy    4.83
sqlite>
```

Figure 9 Merged data in database

Step 5: Automating the Workflow

To automate the ETL workflow, a main() function was implemented to execute all pipeline stages in sequence. The process begins by extracting data from the CSV file, followed by retrieving additional game information from the RAWG API using selected game names.

After extraction, the CSV and API datasets are transformed, cleaned, and merged into a unified dataset. The final cleaned data is then loaded into the SQLite database and exported as a CSV file for visualization and further analysis.

This automated workflow simplifies the ETL execution process and allows the entire pipeline to run using a single command.

```
# --- Run project ---
def main():
    print("Starting ETL Pipeline...")

    # Extract CSV
    raw_csv = extract_csv(CSV_FILE_PATH)

    # Use only first 10 unique names for API to avoid slow requests
    sample_names = (
        raw_csv["NAME"]
        .dropna()
        .astype(str)
        .str.strip()
        .unique()[:10]
        .tolist()
    )

    # Extract API
    raw_api = extract_api(sample_names)

    final_df = transform(raw_csv, raw_api)

    load(final_df, DB_PATH)

    final_df.to_csv("final_games.csv", index=False)

    print("Final CSV saved.")
    print("Done!")


if __name__ == "__main__":
    main()
```

Figure 10 Workflow Automation

4.2 Visualize using BI dashboard

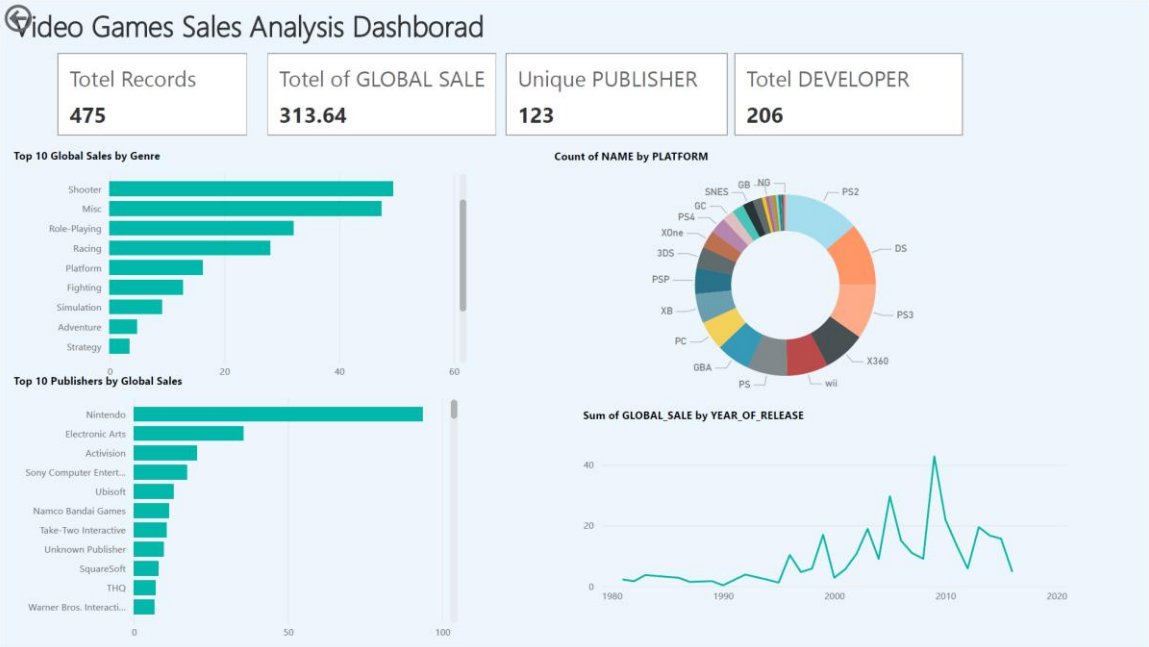


Figure 11 ETL Dashboard

4.3 Performance challenges and solutions

During the ETL implementation, some challenges appeared while retrieving data from the RAWG API, such as slow responses and timeout issues. In addition, the CSV and API sources contained missing values, duplicated records, and inconsistent data types.

These issues were addressed by limiting API requests, cleaning missing values, converting columns into suitable data types, and removing duplicate rows using pandas functions. The final dataset was successfully transformed into a clean and structured format ready for storage and dashboard analysis.

Chapter 5: (Phase 2) Introduction to ELT

5.1 Difference between ELT (Extract, Load, Transform) and ETL (Extract ,Transform , load)

In ETL, the data is collected from different sources, cleaned and transformed first, then loaded into the database or data warehouse.

In ELT, the raw data is collected and loaded directly into the database first. After that, the transformation is done inside the database using SQL.

5.2 Why ELT is used

ELT is used because it is fast, flexible, and suitable for large datasets. It keeps the raw data in the database, so we can check it or transform it again when needed.

In our project, ELT loads the raw video games data first, then cleans and transforms it inside the database for dashboard analysis

5.3 Tools used as Datawarehouse and libraries for ELT

1. Programming Language:

Python: Used to extract and load the raw video games data into the database.

SQL: Used to clean, transform, and query the data inside the data warehouse.

2.Python Libraries:

pandas: Used for reading the CSV file and preparing the raw dataset.

sqlalchemy: Used to connect Python with the PostgreSQL/Neon database.

sqlite3: Used to connect and work with the SQLite database during the extraction and loading process.

3.Data Warehouse:

SQLite: Used as a local database to store and manage raw data before loading it into PostgreSQL.

PostgreSQL (Neon): Used as the central data warehouse where raw data is first loaded, then transformed using SQL queries.

4.Data Transformation:

SQL Queries inside PostgreSQL (Neon): Used to clean and transform the data directly in the data warehouse.

For example, missing values can be handled using COALESCE, text can be cleaned using TRIM.

Chapter 6: ELT Pipeline Design

6.1 Data pipeline architecture (ELT)

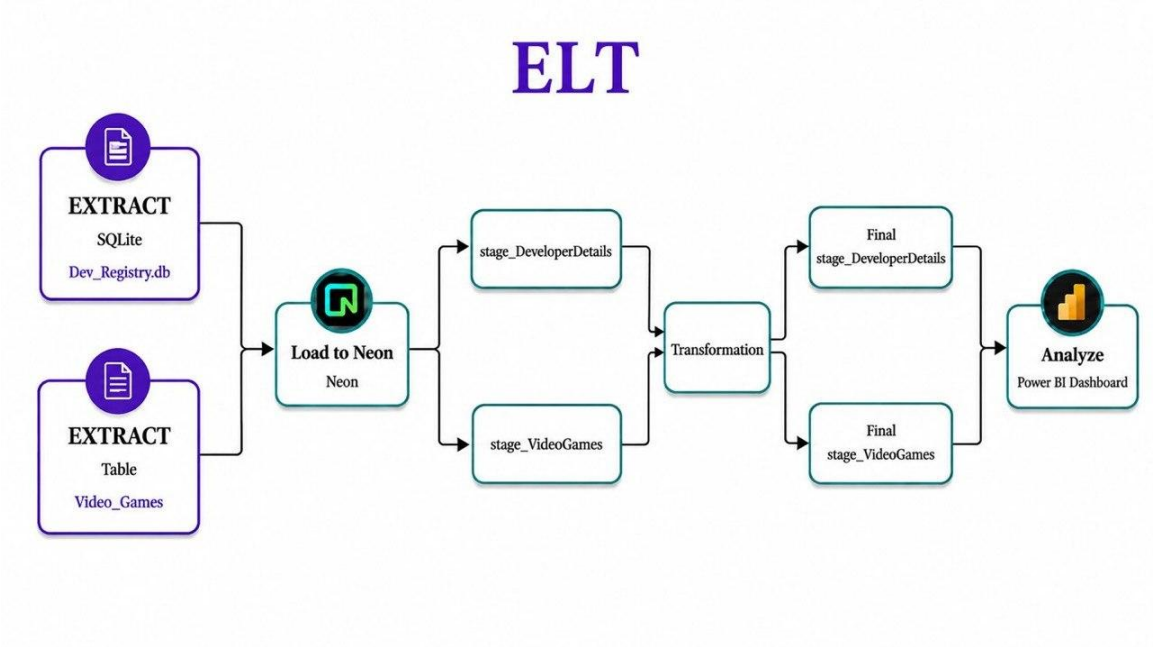


Figure 12 ELT pipeline architecture

6.2 Data sources: Collecting raw data

We collected raw gaming data from two distinct sources to capture both developer metadata and performance metrics:

1. SQLite Database: A database containing developer information such as headquarters and foundation years.

```
c:\sqlite>sqlite3
SQLite version 3.51.3 2026-03-13 10:38:09
Enter ".help" for usage hints.
Connected to a transient in-memory database.
Use ".open FILENAME" to reopen on a persistent database.
sqlite> .open Dev_Registry.db
sqlite> .open
sqlite> .database
main: "" r/w
sqlite> CREATE TABLE IF NOT EXISTS developer_details (
(x1...>     dev_id INTEGER PRIMARY KEY AUTOINCREMENT,
(x1...>     Developer TEXT,
(x1...>     Headquarters TEXT,
(x1...>     Founded_Year INTEGER
(x1...> );
```

Figure 13 Database Creation

```
C:\WINDOWS\system32\CMD
sqlite> SELECT * FROM developer_details;
1|Nintendo|Kyoto, Japan|1889
2|Nintendo|Kyoto, Japan|1889
3|nintendo||1889
4|Ubisoft|Montreuil, France|1986
5|Ubisoft|Montreuil, France|1986
6|Electronic Arts|Redwood City, USA|1982
7|ELECTRONIC ARTS|Redwood City, USA|1982
8|Capcom|Osaka, Japan|1979
9|Capcom||1979
10|Square Enix|Tokyo, Japan|1975
11|Square Enix|Tokyo, Japan|1975
12|Sega|Tokyo, Japan|1960
13|sega|Tokyo, Japan|1960
14|Activision|California, USA|1979
15|Activision||1979
16|Bandai Namco|Tokyo, Japan|2005
17|Bandai Namco|Tokyo, Japan|2005
18|Konami|Tokyo, Japan|1969
19|KONAMI|Tokyo, Japan|1969
20|Take-Two Interactive|New York, USA|1993
21|Take-Two Interactive||1993
22|Sony Interactive Entertainment|San Mateo, USA|1993
23|Sony Interactive Entertainment|San Mateo, USA|1993
24|Bethesda Softworks|Maryland, USA|1986
25|bethesda softworks|Maryland, USA|1986
26|Microsoft Game Studios|Washington, USA|2000
27|Microsoft Game Studios||2000
28|Rockstar Games|New York, USA|1998
29|Rockstar Games|New York, USA|1998
30|Valve Corporation|Washington, USA|1996
31|VALVE CORPORATION|Washington, USA|1996
32|BioWare|Edmonton, Canada|1995
33|BioWare||1995
34|CD Projekt Red|Warsaw, Poland|2002
35|CD Projekt Red|Warsaw, Poland|2002
36|FromSoftware|Tokyo, Japan|1986
37|fromsoftware|Tokyo, Japan|1986
38|Bungie|Washington, USA|1991
39|Insomniac Games|California, USA|1994
40|Insomniac Games|California, USA|1994
41|Naughty Dog|California, USA|1984
42|NAUGHTY DOG|California, USA|1984
```

Figure 14 First 42 row of the data

To extract it, we use the sqlite3 library in Python to connect to the database and pandas to read the SQL query results:

```python
# 1. Extract from SQLite Database
conn_sqlite = sqlite3.connect(r"C:\sqlite\Dev_Registry.db")
df_devs = pd.read_sql_query("SELECT * FROM developer_details", conn_sqlite)
conn_sqlite.close()
```

Figure 15 Extract data from SQLite

2. CSV file (Video_Games.csv): This file contains a dataset of 500

```
# 2. Extract from CSV
csv_path = r"C:\Users\jsdsj\venv\Video_Games.csv"
df_csv = pd.read_csv(csv_path)
```

Figure 16 Extract data from csv file.

video game records, including attributes like Platform, Genre, and Global Sales.

By combining these two sources, we provide a comprehensive view of both developer profiles and game performance for our analysis.

6.3 Load the raw data in datawarehouse

After extracting the data, we established a connection to our cloud data warehouse on Neon PostgreSQL. We used the SQLAlchemy library in Python to create an engine using our Neon connection string.

- stage_DeveloperDetails: for the data extracted from the SQLite database.
- stage_VideoGames: for the dataset loaded from the CSV file.

By using the to_sql function with our Neon engine, we ensured that the raw data was successfully migrated from our local environment to the cloud

```
# 3. Connect to Neon PostgreSQL
NEON_URL = "postgresql://neondb_owner:npg_g3z8cRGSVlmh@ep-odd-darkness-ap13lky9-pooler.c-7.us-east-1.aws.neon.tech/r
engine = create_engine(NEON_URL)

# 4. Load both into Neon (Staging Tables)
df_devs.to_sql("stage_DeveloperDetails", engine, if_exists="replace", index=False)
df_csv.to_sql("stage_VideoGames", engine, if_exists="replace", index=False)

print("Data loaded into the staging tables successfully!")
```

Figure 17 stablishing connection to Neon cloud data warehouse using SQLAlchemy.

```
(.venv) PS C:\Users\jsdsj\venv> python project.py
Data loaded into the staging tables successfully!
```

Figure 18 output confirming successful script execution.

for centralized storage.

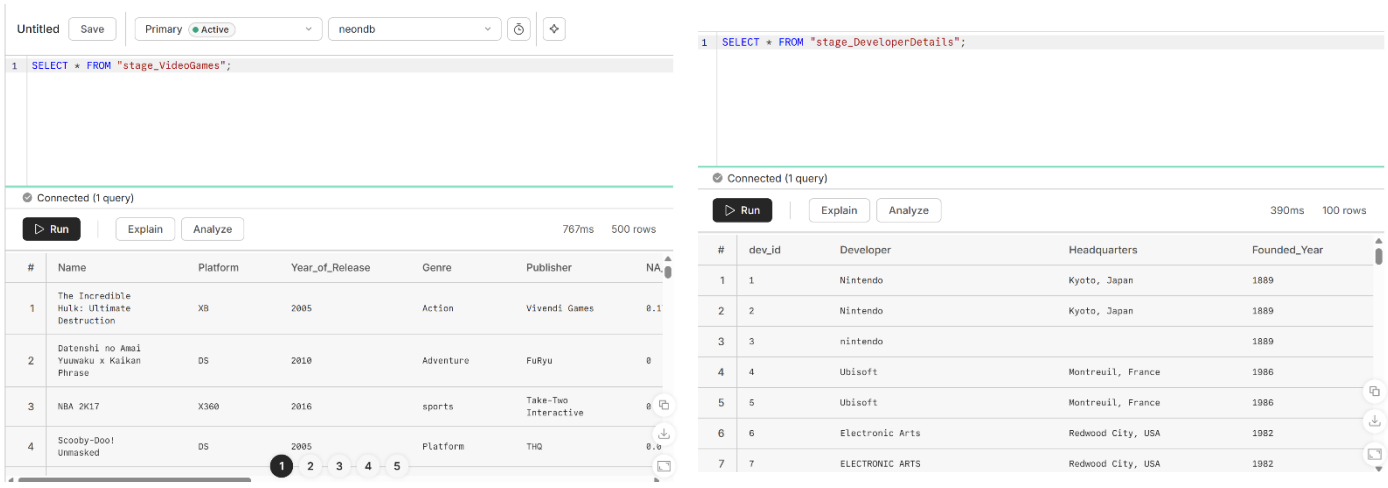


Figure 19 Staging tables

6.4 Transform: SQL based transformation

After extracting and loading the raw data into the Neon PostgreSQL staging tables, , we applied SQL cleaning and transformation directly within the Neon PostgreSQL environment. Using the SQL Editor, we moved the data from staging tables to final tables while ensuring data quality through specific SQL functions.

Table 1: final_VideoGames:

This final table was created in Neon to store the cleaned version of the video games dataset.

```
DROP TABLE IF EXISTS final_VideoGames;
CREATE TABLE final_VideoGames (
    game_id SERIAL PRIMARY KEY,
    Name TEXT,
    Platform TEXT,
    Year_of_Release INTEGER,
    Genre TEXT,
    Publisher TEXT,
    Global_Sales REAL
);
```

Figure 20 Final Table for VideoGames

Transformations performed:

SELECT DISTINCT: Used to remove any duplicate game records.

TRIM(): Used to remove extra spaces from the game Names and Platforms.

UPPER(): Used on the Platform column to standardize it (e.g., PS4, PC).

CAST(... AS INTEGER): Used to convert the Year_of_Release into a numeric format for sorting.

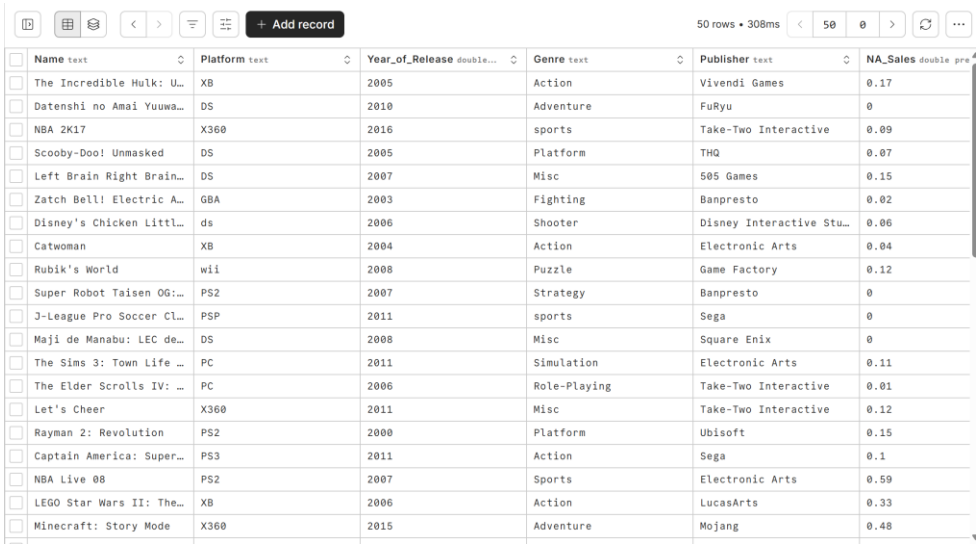
COALESCE(): This function was used for multiple purposes:

- To replace missing Year_of_Release values with 0.
- To handle missing Genre entries by replacing them with the default category 'Miscellaneous'.
- To replace empty Publisher fields with 'Unknown' and null Global_Sales with 0.0.

WHERE "Name" IS NOT NULL: Used to ensure no incomplete records without names are inserted.

```
INSERT INTO final_videogames (name, platform, year_of_release, genre, publisher, global_sales)
SELECT DISTINCT
    TRIM("Name") AS name,
    UPPER(TRIM("Platform")) AS platform,
    COALESCE(CAST("Year_of_Release" AS INTEGER), 0) AS year_of_release,
    COALESCE("Genre", 'Miscellaneous') AS genre,
    COALESCE("Publisher", 'Unknown') AS publisher,
    COALESCE("Global_Sales", 0.0) AS global_sales
FROM "stage_VideoGames"
WHERE "Name" IS NOT NULL
```

Figure 21 Transformations for videoGames



+ Add record — 50 rows • 308ms — 50 — 0

Name text	Platform text	Year_of_Release double...	Genre text	Publisher text	NA_Sales double pre
The Incredible Hulk: U…	XB	2005	Action	Vivendi Games	0.17
Datenshi no Amai Yuuwa…	DS	2010	Adventure	FuRyu	0
NBA 2K17	X360	2016	sports	Take-Two Interactive	0.09
Scooby-Doo! Unmasked	DS	2005	Platform	THQ	0.07
Left Brain Right Brain…	DS	2007	Misc	505 Games	0.15
Zatch Bell! Electric A…	GBA	2003	Fighting	Banpresto	0.02
Disney's Chicken Littl…	ds	2006	Shooter	Disney Interactive Stu…	0.06
Catwoman	XB	2004	Action	Electronic Arts	0.04
Rubik's World	wii	2008	Puzzle	Game Factory	0.12
Super Robot Taisen OG:…	PS2	2007	Strategy	Banpresto	0
J-League Pro Soccer Cl…	PSP	2011	sports	Sega	0
Maji de Manabu: LEC de…	DS	2008	Misc	Square Enix	0
The Sims 3: Town Life …	PC	2011	Simulation	Electronic Arts	0.11
The Elder Scrolls IV: …	PC	2006	Role-Playing	Take-Two Interactive	0.01
Let's Cheer	X360	2011	Misc	Take-Two Interactive	0.12
Rayman 2: Revolution	PS2	2000	Platform	Ubisoft	0.15
Captain America: Super…	PS3	2011	Action	Sega	0.1
NBA Live 08	PS2	2007	Sports	Electronic Arts	0.59
LEGO Star Wars II: The…	XB	2006	Action	LucasArts	0.33
Minecraft: Story Mode	X360	2015	Adventure	Mojang	0.48

Figure 22 before appley transformations

game_id serial	name text	platform text	year_of_release integer	genre text	publisher text
1	TV Anime Idolm@ster: C…	PS3	2016	Misc	Unknown
2	Rhythm Heaven: The Bes…	3DS	2015	Misc	Nintendo
3	All Grown Up!: Game Bo…	GBA	2004	misc	Unknown
4	From Russia With Love	XB	2005	Action	Electronic Arts
5	Super Monkey Ball Delu…	XB	2005	Misc	Sega
6	Fantastic 4	XB	2005	Action	Activision
7	Hulk	XB	2003	Action	Universal Intera
8	Desktop Tower Defense	DS	2009	Strategy	THQ
9	WSC Real 11: World Sno…	PS3	2011	Sports	Koch Media
10	Goat Simulator	XONE	2016	Simulation	Koch Media
11	Pocket Pets	DS	2007	Simulation	O3 Entertainment
12	Just Dance 4	X360	2012	Misc	Ubisoft
13	Cabela's North America…	PSP	2010	Sports	Activision
14	Teenage Mutant Ninja T…	DS	2009	Action	Ubisoft
15	Black Bass with Blue M…	PS	1999	Sports	Starfish
16	Pirates of the Caribbe…	PSP	2006	Adventure	Disney Interacti
17	Bass Hunter 64	N64	1999	Sports	Take-Two Interac
18	Monster Hunter Frontie…	X360	2010	Role-Playing	Capcom
19	NBA 2K11	X360	2010	Action	Take-Two Interac
20	Kirby's Adventure	NES	1993	Platform	Nintendo

Figure 23 after appley transformations

- Table 2: final_DeveloperDetails:

  A second transformation was applied to the developers' data to ensure consistency across the database.

```
1  DROP TABLE IF EXISTS final_DeveloperDetails;
2
3  CREATE TABLE final_DeveloperDetails (
4      id SERIAL PRIMARY KEY,
5      Developer TEXT,
6      Headquarters TEXT,
7      Founded_Year INTEGER
8  );
```

Figure 24 Final table for DeveloperDetails

Transformations performed:

INITCAP(): Used to capitalize the first letter of each Developer name.

TRIM(): Used to clean the developer names from leading or trailing spaces.

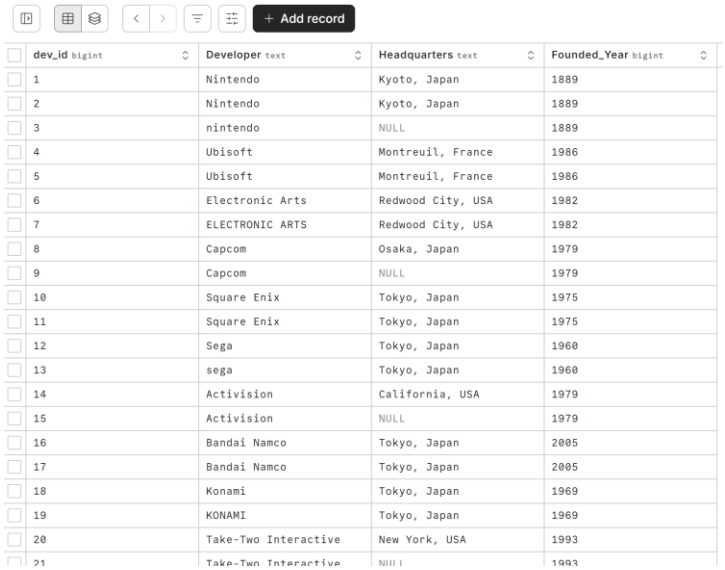
COALESCE(): Used to replace missing Headquarters data with the value 'Unknown'.

SELECT DISTINCT: Used to ensure each developer is only listed once in the final table.

WHERE "Developer" IS NOT NULL: Used to ensure no incomplete records without Developer are inserted.

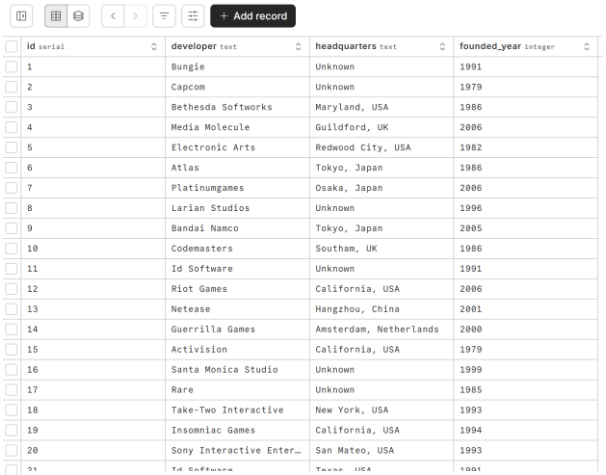
```sql
INSERT INTO final_developerdetails (developer, headquarters, founded_year)
SELECT DISTINCT
    INITCAP(TRIM("Developer")) AS developer,
    COALESCE("Headquarters", 'Unknown') AS headquarters,
    "Founded_Year"
FROM "stage_DeveloperDetails"
WHERE "Developer" IS NOT NULL
```

Figure 25 Transformations for DeveloperDetails



dev_id bigint	Developer text	Headquarters text	Founded_Year bigint
1	Nintendo	Kyoto, Japan	1889
2	Nintendo	Kyoto, Japan	1889
3	nintendo	NULL	1889
4	Ubisoft	Montreuil, France	1986
5	Ubisoft	Montreuil, France	1986
6	Electronic Arts	Redwood City, USA	1982
7	ELECTRONIC ARTS	Redwood City, USA	1982
8	Capcom	Osaka, Japan	1979
9	Capcom	NULL	1979
10	Square Enix	Tokyo, Japan	1975
11	Square Enix	Tokyo, Japan	1975
12	Sega	Tokyo, Japan	1960
13	sega	Tokyo, Japan	1960
14	Activision	California, USA	1979
15	Activision	NULL	1979
16	Bandai Namco	Tokyo, Japan	2005
17	Bandai Namco	Tokyo, Japan	2005
18	Konami	Tokyo, Japan	1969
19	KONAMI	Tokyo, Japan	1969
20	Take-Two Interactive	New York, USA	1993
21	Take-Two Interactive	NULL	1993

Figure 26 before appley transformations



id serial	developer text	headquarters text	founded_year integer
1	Bungie	Unknown	1991
2	Capcom	Unknown	1979
3	Bethesda Softworks	Maryland, USA	1986
4	Media Molecule	Guildford, UK	2006
5	Electronic Arts	Redwood City, USA	1982
6	Atlas	Tokyo, Japan	1986
7	Platinumgames	Osaka, Japan	2006
8	Larian Studios	Unknown	1996
9	Bandai Namco	Tokyo, Japan	2005
10	Codemasters	Southam, UK	1986
11	Id Software	Unknown	1991
12	Riot Games	California, USA	2006
13	Netease	Hangzhou, China	2001
14	Guerrilla Games	Amsterdam, Netherlands	2000
15	Activision	California, USA	1979
16	Santa Monica Studio	Unknown	1999
17	Rare	Unknown	1985
18	Take-Two Interactive	New York, USA	1993
19	Insomniac Games	California, USA	1994
20	Sony Interactive Enter…	San Mateo, USA	1993
21	Id Software	Texas, USA	1991

Figure 27 after appley transformations

Chapter 7: Implementation & Results

7.1 Execution of ELT workflow

The ELT pipeline was executed successfully based on the architecture and workflow designed in Chapter 6. The implementation followed three main stages: extraction, loading, and transformation. The workflow was developed using Python, PostgreSQL (Neon), SQLAlchemy, pandas, sqlite3, and SQL queries executed in PostgreSQL (Neon).

Tools and Technologies Used:

Python (Visual Studio Code): Used to implement and automate the ELT workflow.

Pandas: Used for extracting and reading data from CSV files and SQLite databases.

sqlite3: Used to connect with the local SQLite database during extraction.

SQLAlchemy: Used to establish a connection between Python and Neon PostgreSQL.

PostgreSQL (Neon) : Used as the cloud data warehouse for staging, transformation, and final storage using SQL queries.

7.1.1 Extract

In the extraction phase, raw gaming data was collected from two different sources:

A local SQLite database containing developer information such as developer names, headquarters, and foundation years.

A CSV file named Video_Games.csv containing video game records including platform, genre, publisher, and global sales.

The extraction process was implemented using:

sqlite3.connect() to connect to the SQLite database.

pd.read_sql_query() to extract database records into pandas DataFrames.
pd.read_csv() to load the CSV dataset into Python.

7.1.3 Load

After extraction, the raw datasets were loaded directly into Neon PostgreSQL staging tables using SQLAlchemy and the pandas to_sql() function.

The following staging tables were created:
stage_DeveloperDetails
stage_VideoGames
The loading phase migrated the raw data from the local environment into the cloud data warehouse successfully. Console output messages confirmed that the loading process completed without errors.

7.1.4 Transform (SQL)

After loading the raw data into PostgreSQL (Neon) staging tables, SQL transformations were executed directly inside Neon PostgreSQL using internal dieter.
Several SQL functions were applied to clean and standardize the data, including:
SELECT DISTINCT to remove duplicate records.
TRIM() to clean extra spaces from text columns.
UPPER() to standardize platform names.
COALESCE() to replace missing values with default values.
CAST(... AS INTEGER) to convert release years into numeric format.
The cleaned data was then inserted into the final tables:
final_VideoGames
final_DeveloperDetails

7.1.5 Execution Result

The ELT workflow completed successfully without runtime errors.

The final PostgreSQL tables contained cleaned, structured, and standardized gaming data ready for dashboard analysis and visualization.

The execution results confirmed that:

Data extraction completed successfully.

Raw data was loaded correctly into PostgreSQL staging tables.

SQL transformations improved data quality and consistency.

Final tables were generated successfully for reporting and BI dashboard visualization.

7.2 Visualize using BI dashboard

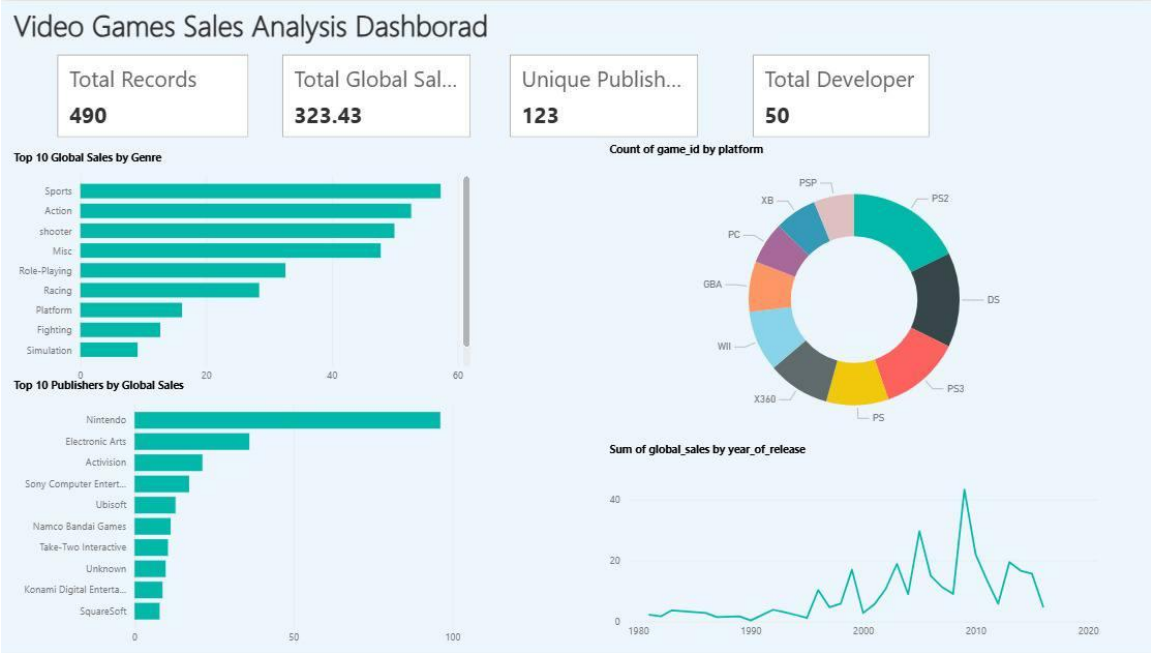


Figure 28 Final BI Dashboard visualizing the processed data

7.3 Automation of ELT pipeline using Task scheduler

I used ChatGPT promote to help understand Task scheduler and create one

Figure 29 AI integration

Explain Task Scheduler in Windows, and how to create it using task schedule? explain all steps for scheduling Python script automatically every day at 11:59 PM for ELT/data pipeline project.

What is Windows Task Scheduler?

Windows Task Scheduler is a built in tool in Windows that allows users to run programs or scripts automatically at a specific time or event. It is very useful in ELT or data pipeline projects because it can run Python scripts without opening them manually every day.

For example in ELT projects Task Scheduler can:

- Run data extraction scripts every night
- Update databases automatically
- Generate reports daily
- Automate data cleaning and transformation

In our case we will use it to run a Python script automatically every day at 11:59 PM.

Steps to Schedule a Python Script in Task Scheduler

Step 1: Prepare the Python Script

Make sure your Python script works correctly before scheduling it.

Example:

Python

```python
print("ELT Pipeline Running")
```

Save the file for example as:

```
pipeline.py
```

Example location:

```
C:\ELT_Project\pipeline.py
```

Step 2: Find Python.exe Path

Task Scheduler needs the location of Python.

How to find it:

1. Open Command Prompt
2. Type:

```bash
where python
```

Example output:

```
C:\Users\Sara\AppData\Local\Programs\Python\Python312\python.exe
```

Copy this path because we will use it later.

Step 3: Open Task Scheduler

1. Press Windows Search
2. Type:

```
Task Scheduler
```

3. Open the application

Step 4: Create a Basic Task

On the right side click:

```
Create Basic Task
```

Step 5: Name the Task

Write:

- Name:

```
ELT Pipeline Automation
```

- Description:

```
Run Python ELT pipeline every day automatically
```

Then click:

```
Next
```

Step 6: Choose Trigger

Trigger means when the task will run.

1. Select:

```
Daily
```

2. Click:

```
Next
```

Step 7: Set Time

Choose:

```
11:59:00 PM
```

Set:

- Recur every:

```
1 day
```

Then click:

```
Next
```

Step 8: Choose Action

Select:

```
Start a Program
```

Then click:

```
Next
```

Step 9: Add Python Program

In Program/script:

Paste Python path.

Example:

```
C:\Users\Sara\AppData\Local\Programs\Python\Python312\python.exe
```

In Add arguments:

Write the script path inside quotes.

Example:

```
"C:\ELT_Project\pipeline.py"
```

In Start in:

Write the folder location only.

Example:

```
C:\ELT_Project
```

Then click:

```
Next
```

Step 10: Finish the Task

Check the information then click:

```
Finish
```

Now the Python ELT script will run automatically every day at 11:59 PM.

How to Test the Task

Before waiting until 11:59 PM you should test it.

1. Open Task Scheduler
2. Find your task
3. Right click the task
4. Press:

```
Run
```

If everything is correct the Python script will execute automatically.

My implementation **Task Scheduler**.

• Scheduling: I set a daily trigger to run the Python script (Project.py) at a specific time.

• Configuration: I used the Python interpreter from the virtual environment (venv) to ensure all libraries work correctly.

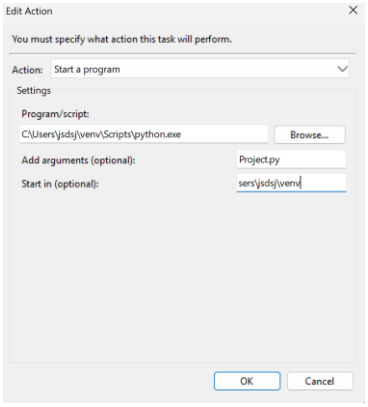


Figure 30 Configuring Task Scheduler action with the virtual environment.

• Verification: The task was successful, showing the status (0x0), which confirms that the data was transferred to Neon PostgreSQL effectively.

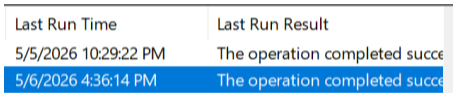


Figure 31 Task Scheduler history confirming successful run .

Figure 32 database updated successfully following the automated run

7.4 Comparison of ETL vs. ELT (speed, scalability, efficiency)

Performance Compariso	ETL Pipeline	ELT Pipeline
Speed	14.529 sec	15.03 sec
Scalability	Less scalable	More scalable
Efficiency	Lower	Higher

Table 1 ETL vs ELT

Speed

ETL: The ETL pipeline completed in 14.529 seconds, as local pandas transformations and API merging were performed before loading into SQLite.

ELT: The ELT pipeline took 15.03 seconds for extraction and loading only. The slight increase is due to network latency when connecting to the Neon PostgreSQL cloud warehouse.

Scalability

ETL: Less scalable since it depends on local RAM to process the full dataset, which may not handle large or complex datasets efficiently as data volume grows.

ELT: More scalable as it leverages the power of cloud-based PostgreSQL (Neon) to handle large datasets natively without overloading the local machine.

Efficiency

ETL: Lower efficiency due to multiple explicit transformation steps written in Python, which increases code complexity and maintenance overhead.

ELT: Highly efficient because Python handles only extraction and loading, while all transformations are executed inside Neon using SQL, reducing code complexity and enabling easier auditing and modification.

7.5 Conclusion

This project successfully demonstrated the design and implementation of both **ETL** and **ELT** pipelines applied to a video games dataset. By integrating data from a CSV file, the **RAWG API**, and a local **SQLite** database, the project provided a practical comparison of how architectural choices impact performance and scalability.

In the **ETL phase**, data was transformed using **Python and pandas** before being loaded into SQLite. While this approach allowed for granular control over data quality, it faced limitations in speed and scalability due to local memory and CPU constraints. Conversely, the **ELT phase** leveraged the power of the **Neon PostgreSQL** cloud warehouse. By loading raw data directly into staging tables and performing transformations using **SQL**, the pipeline achieved higher efficiency, easier maintainability, and better auditability.

The automation of these pipelines using **Windows Task Scheduler** further ensured a reliable, intervention-free data flow. Ultimately, this project reinforces that while ETL is effective for controlled, smaller workloads, ELT is the superior choice for modern, scalable cloud environments. This hands-on experience highlights the critical importance of selecting the right architecture based on specific use cases and data requirements in real-world data engineering.

7.6 References (IEEE format)

[1] IBM, "What is Data Engineering?" Available: https://www.ibm.com/topics/data-engineering

[2] Microsoft, "ETL vs ELT: What's the Difference?" Available: https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-etl

[3] United Nations, "Sustainable Development Goals," United Nations. Available: https://sdgs.un.org/goals. Accessed: May 9, 2026.

[4] AWS, "Batch Processing Explained." Available: https://aws.amazon.com/what-is/batch-processing/